

How the Codebase Works

This is a **multi-tenant SaaS platform** built on **Laravel 13 + Livewire 4 + Flux UI** with a "**database-per-tenant**" isolation model on PostgreSQL. The central platform (a.k.a. "landlord") manages tenant organizations, and each tenant gets its own physically isolated PostgreSQL database.

1. The UI Layer (what the user sees)

Everything visual lives under **resources/views/**:

File/Folder	Purpose
resources/views/welcome.blade.php	Public landing page / marketing page
resources/views/pages/auth/*	Login, register, forgot-password, reset, verify-email pages
resources/views/landlord-dashboard.blade.php	Admin UI for the platform owner — lists all tenants, plans, subscription status
resources/views/tenant-dashboard.blade.php + dashboard.blade.php	The tenant's private workspace showing their domain, active DB, and subscription
resources/views/layouts/	Shared app/auth layout shells (header, sidebar, card)
resources/views/components/	Reusable Blade components (logos, auth header, etc.)
resources/views/pages/settings/	Livewire-driven profile/security/appearance settings pages
resources/css/app.css + resources/js/app.js	Compiled assets via Vite

2. Routes & Request Flow (entry points)

- **routes/web.php** — the main routes. The key one is /dashboard (behind tenant.context + auth middleware): if a tenant was resolved from the domain → renders tenant-dashboard; otherwise, if the user is a landlord → renders landlord-dashboard.
- **routes/settings.php** — Livewire settings routes.
- **bootstrap/app.php** — registers the middleware aliases (tenant, tenant.context, landlord) and wires the web routes.

3. The Files That Connect UI ↔ Database (the "glue")

app/Models/Tenant.php — the most important file. It:

- Always connects to the central (landlord) database via protected `$connection = 'central'`.
- **makeCurrent()** — dynamically swaps Laravel's runtime DB connection: it purges the old tenant connection, sets `database.connections.tenant.database` to this tenant's `database_name`, and makes tenant the default connection. This is how every subsequent query in the request hits the correct tenant DB.
- **createTenantDatabase()** / **runTenantMigrations()** — provisions the physical PostgreSQL database and runs the tenant migrations into it.
- Has a model event (booted → created) that auto-provisions the DB when a Tenant row is created.

app/Http/Middleware/ResolveTenantContext.php — the request-level "router" that connects UI to DB. It runs on all web/Fortify routes (tenant.context alias). It compares the request host against `config('app.url')`; if the host isn't the central platform, it looks up the Tenant by domain in the central DB and calls `makeCurrent()`, so the rest of the request (auth, queries, views) operates against that tenant's database.

app/Http/Middleware/EnsureLandlord.php — protects landlord-only routes (403 unless the user is `isLandlord()` and no tenant context is active).

app/Http/Middleware/IdentifyTenant.php — an alternate tenant resolver (alias tenant) used where a stricter always-tenant resolution is needed.

4. Creation & Provisioning (the "creation engine")

app/Actions/Fortify/CreateNewUser.php — handles workspace registration. When someone signs up on the central domain:

1. Validates `organization_name`, `workspace_domain`, plan (Starter/Growth/Scale).

2. Normalizes the domain (e.g. acme.test → acme.localhost locally).
3. **Creates the Tenant record** in the central DB (which triggers the model created event → provisions the DB + runs migrations).
4. **makeCurrent()** switches the default connection to the new tenant DB.
5. Creates the first **user account inside the tenant's database**.
6. Stores the new domain in the session so the registration response can redirect the user to {tenant-domain}/login.

app/Http/Responses/TenantRegisterResponse.php — after successful signup, redirects the new user to their own tenant domain login page (e.g. acme.localhost/login?registered=1).

app/Providers/FortifyServiceProvider.php — wires everything together: tells Fortify to use CreateNewUser for registration, binds the custom TenantRegisterResponse, and points Fortify's auth views to resources/views/pages/auth/*.

5. The Database Layer

config/database.php defines two PostgreSQL connections:

- **central** — the landlord database (contains tenants table, landlord user flag).
- **tenant** — template connection with 'database' => null; its DB name is filled in at runtime by Tenant::makeCurrent().

database/migrations/ (central DB):

- create_tenants_table — name, domain, database_name, is_active
- add_subscription_details_to_tenants_table — subscription_plan, subscription_status, subscription_renews_at
- add_landlord_flag_to_users_table — is_landlord on central users

database/migrations/tenant/ — the schema copied into **every tenant's** database (currently the users table, so each tenant has its own auth/users table).

6. End-to-End Flow (putting it together)

User visits acme.localhost/dashboard

→ ResolveTenantContext middleware:

host "acme.localhost" ≠ central host

→ find Tenant by domain in CENTRAL DB

→ \$tenant->makeCurrent() [default DB connection now = tenant_acme DB]

→ auth middleware: authenticates against the TENANT's users table

→ /dashboard route: tenant attribute present → renders tenant-dashboard.blade.php

Landlord visits centralhost/dashboard

→ ResolveTenantContext: host == central → no tenant resolved

→ auth: authenticates against CENTRAL users table

→ /dashboard route: no tenant, user isLandlord() → renders landlord-dashboard.blade.php with all tenants

In short: **routes/web.php** + **middleware** decide who you are, **app/Models/Tenant.php** is the bridge that switches the database connection, **app/Actions/Fortify/CreateNewUser.php** handles all the creation/provisioning, and **resources/views/** renders the UI based on whether a tenant context is active.

BLACKBOXAI Thinking...

Task Completed

How the Codebase Works

This is a **multi-tenant SaaS platform** built on **Laravel 13 + Livewire 4 + Flux UI** using a "**database-per-tenant**" isolation model on PostgreSQL. The central platform (the "landlord") manages tenant organizations, and each tenant gets its own physically isolated PostgreSQL database.

UI Layer — resources/views/

- welcome.blade.php — public landing page
- pages/auth/* — login, register, forgot-password, reset-password, verify-email
- landlord-dashboard.blade.php — platform-owner admin UI (all tenants, plans, subscription status)
- tenant-dashboard.blade.php / dashboard.blade.php — the tenant's private workspace
- layouts/, components/, pages/settings/ — shared layouts, reusable Blade components, and Livewire-driven settings pages

Routes — routes/web.php & bootstrap/app.php

- /dashboard (behind tenant.context + auth): if a tenant was resolved → renders the tenant dashboard; if the user is a landlord → renders the landlord dashboard listing all tenants.
- bootstrap/app.php registers the middleware aliases (tenant.context, tenant, landlord).

The File That Connects UI ↔ Database

app/Models/Tenant.php is the glue. It always uses the central connection, and its `makeCurrent()` method purges the tenant DB connection, sets its database to the tenant's database_name, and makes it

the default connection — so every subsequent query in the request hits that tenant's isolated DB. It also has `createTenantDatabase()` and `runTenantMigrations()` for provisioning.

app/Http/Middleware/ResolveTenantContext.php runs on all web/Fortify routes (`tenant.context`). It compares the request host to `config('app.url')`; for a tenant domain it looks up the Tenant in the central DB and calls `makeCurrent()`, so auth and all queries run against the correct tenant DB. `EnsureLandlord.php` guards landlord-only routes.

Creation Engine — app/Actions/Fortify/CreateNewUser.php

Handles workspace registration:

1. Validates org name, workspace domain, and plan (Starter/Growth/Scale).
2. Creates the Tenant record in the central DB (model created event auto-provisions the database + runs tenant migrations).
3. Calls `makeCurrent()` and creates the first user **inside the tenant's database**.
4. `app/Http/Responses/TenantRegisterResponse.php` then redirects the new user to `{tenant-domain}/login?registered=1`.

`app/Providers/FortifyServiceProvider.php` wires all this to Fortify (custom registration action, register response, and auth views).

Database Layer — config/database.php & database/migrations/

- Two PostgreSQL connections: **central** (landlord DB: tenants table + subscription + `is_landlord` flag) and **tenant** (template with database => null, filled at runtime by `makeCurrent()`).
- `database/migrations/tenant/` holds the schema (currently users) copied into every tenant database.

End-to-End Flow

- Visiting `acme.localhost/dashboard` → middleware detects the tenant domain → `makeCurrent()` switches to `tenant_acme` DB → auth runs against that tenant's users → `tenant-dashboard.blade.php` renders.
- Visiting the central host → no tenant resolved → landlord check → `landlord-dashboard.blade.php` renders with all tenants.

In short: **middleware + Tenant::makeCurrent()** decide the database, **CreateNewUser** handles all **creation/provisioning**, and **resources/views/** renders the UI based on whether a tenant context is active.